BITÁCORA DE LABORATORIO

Práctica: Laboratorio 1.1: Aplicación básica con Spring Boot, endpoints REST, Actuator, perfiles y DevTools

Asignatura: Programación Avanzada

Estudiante: Andres Jahir Ushiña Sosa

Fecha: 16/10/2026

1. Entorno

Java y Maven: (salidas de java -version y .\mvnw -v)

Salida de java --version

Plaintext
openjdk 21.0.12.1 2026-08-18 LTS
OpenJDK Runtime Environment Temurin-21.0.12.1+1 (build 21.0.12.1+1-LTS)
OpenJDK 64-Bit Server VM Temurin-21.0.12.1+1 (build 21.0.12.1+1-LTS, mixed mode, sharing)

Salida de .\mvnw.cmd -v

Plaintext
Apache Maven 3.10.0 (c43a36b8d67be7e0805a411bc0898af1a51f5472)
Maven home: C:\Users\Admin\.m2\wrapper\dists\apache-maven-3.10.0\2aaba761688b1a6ea5a46e7e5f4d5406ed7a75ee8da95e5e3079863a23461c9b
Java version: 21.0.12.1, vendor: Eclipse Adoptium, runtime: C:\Program Files\Eclipse Adoptium\jdk-21.0.12.101-hotspot
Default locale: es_EC, platform encoding: UTF-8, time zone: America/Guayaquil
OS name: "windows 11", version: "10.0", arch: "amd64", family: "windows"

2. Reinicio con DevTools

- **Cambio realizado:**

  Con la aplicación en ejecución en el puerto 8080, se modificó el valor de la propiedad app.mensaje en el archivo src/main/resources/application.properties, concatenando la cadena " - Hola que hace":

  Properties

  app.mensaje=Aplicación Spring Boot en funcionamiento - Hola que hace

  Se guardaron los cambios y se activó la compilación del proyecto desde el IDE IntelliJ IDEA.

- **Evidencia del reinicio (registro):**

  DevTools detectó el cambio en el classpath y desencadenó un reinicio automático en caliente a través del hilo restartedMain sin detener la ejecución de la JVM ni requerir un reinicio manual:

  Plaintext

  ```
  2026-10-16T12:11:11.101-05:00  INFO 16548 --- [lab] [  restartedMain] .rClassPathChangeChangedEventListener : Restarting application due to class path change
  2026-10-16T12:11:11.103-05:00  INFO 16548 --- [lab] [        Thread-1] o.s.boot.SpringApplication               : Commencing graceful shutdown
  2026-10-16T12:11:11.126-05:00  INFO 16548 --- [lab] [tomcat-shutdown] o.s.boot.tomcat.GracefulShutdown         : Graceful shutdown complete
  ...
  2026-10-16T12:11:12.290-05:00  INFO 16548 --- [lab] [  restartedMain] o.s.boot.tomcat.TomcatWebServer          : Tomcat started on port 8080 (http) with context path '/'
  2026-10-16T12:11:12.297-05:00  INFO 16548 --- [lab] [  restartedMain] com.espe.lab.LabApplication              : Started LabApplication in 1.182 seconds
  ```

- **Respuesta antes y después:**
  - **Antes:** Petición GET a http://localhost:8080/api/saludo:

    JSON

    ```
    {
      "mensaje": "Aplicación Spring Boot en funcionamiento - ESPE",
      "perfil": "default"
    }
    ```

  - **Después:** Petición GET a http://localhost:8080/api/saludo:

    JSON

    ```
    {
      "mensaje": "Aplicación Spring Boot en funcionamiento - Hola que hace",
      "perfil": "default"
    }
    ```

3. Incidencias Técnicas Documentadas

Incidencia 1: Incompatibilidad de versión del compilador Java (Fase 1 / 5)

- **Síntoma observado:**

Al intentar compilar y ejecutar el proyecto con el comando Maven Wrapper (.\mvnw.cmd spring-boot:run), el proceso falló inmediatamente antes de iniciar el contexto de Spring.

- **Mensaje de error:**

  Plaintext

  [ERROR] Failed to execute goal org.apache.maven.plugins:maven-compiler-plugin:3.15.0:compile (default-compile) on project lab: Fatal error compiling: error: release version 21 not supported -> [Help 1]

- **Causa identificada:**

  El archivo pom.xml requiere compilar con bytecode de Java 21 (release 21), pero la variable de entorno de sesión de Maven no apuntaba al JDK 21 instalado, utilizando un JDK anterior o una variable JAVA_HOME no configurada hacia el directorio de Eclipse Adoptium.

- **Solución aplicada:**

  Se identificó la ruta absoluta del JDK 21 mediante PowerShell ((Get-Command java).Source) y se redefinieron las variables de entorno para la sesión activa:

  PowerShell

  $env:JAVA_HOME = "C:\Program Files\Eclipse Adoptium\jdk-21.0.12.101-hotspot"
  $env:Path = "$env:JAVA_HOME\bin;" + $env:Path

- **Verificación posterior:**

  Se ejecutó .\mvnw.cmd -v corroborando que Maven detecta Java version: 21.0.12.1. Posteriormente se lanzó el comando de compilación, obteniendo el estado BUILD SUCCESS.

Incidencia 2: Error de versión de clase en tiempo de ejecución (UnsupportedClassVersionError)

- **Síntoma observado:**

  Al ejecutar el aplicativo en una nueva ventana de terminal, el proceso de Maven finalizó abruptamente con código de salida 1 al intentar cargar la clase principal LabApplication.

  PDF

- **Mensaje de error:**

  Plaintext

Error: Se ha producido un error de enlace al cargar la clase principal com.espe.lab.LabApplication
java.lang.UnsupportedClassVersionError: com/espe/lab/LabApplication has been compiled by a more recent version of the Java Runtime (class file version 65.0), this version of the Java Runtime only recognizes class file versions up to 61.0
[ERROR] Failed to execute goal org.springframework.boot:spring-boot-maven-plugin:4.1.1:run (default-cli) on project lab: Process terminated with exit code: 1

- **Causa identificada:**

  El código fue compilado previamente con Java 21 (versión de clase 65.0), pero la nueva sesión de PowerShell ejecutó el proceso con un runtime Java 17 (versión de clase 61.0) debido a la ausencia de persistencia de JAVA_HOME en las variables del sistema de Windows.

- **Solución aplicada:**

  Se exportaron nuevamente las variables de entorno en la terminal actual hacia JDK 21 y se configuró permanentemente JAVA_HOME en las Variables de Entorno del Sistema de Windows (sysdm.cpl), priorizando %JAVA_HOME%\bin en la variable Path.

- **Verificación posterior:**

  Se reanudó la ejecución con .\mvnw.cmd spring-boot:run "-Dspring-boot.run.profiles=dev", logrando el arranque exitoso de la aplicación en el puerto 8081 sin errores de enlace.

Incidencia 3: Conflicto de puertos por ejecución concurrente (Fase 6 - Error controlado)

- **Síntoma observado:**

  Con una instancia de la aplicación ejecutándose en primer plano en el puerto 8081, se intentó iniciar una segunda instancia simultánea apuntando al mismo puerto desde otra terminal. El arranque falló de manera controlada.

- **Mensaje de error:**

  Plaintext

  ****
  APPLICATION FAILED TO START
  ****

  Description:

  Web server failed to start. Port 8081 was already in use.

Action:

Identify and stop the process that's listening on port 8081 or configure this application to listen on another port.

- **Causa identificada:**

  Colisión de sockets a nivel de red. El sistema operativo no permite que dos procesos diferentes enlacen el mismo socket TCP (0.0.0.0:8081) simultáneamente bajo el protocolo HTTP.

- **Solución aplicada:**

  Se identificó el proceso activo y se detuvo la primera instancia mediante Ctrl + C en la terminal inicial. Alternativamente, se ejecutó la aplicación configurando un puerto libre mediante propiedades de perfil.

- **Verificación posterior:**

  Tras liberar el puerto 8081, se repitió el comando de ejecución y Tomcat inicializó correctamente sin colisiones:

  Plaintext

  Tomcat started on port 8081 (http) with context path '/'

4. Uso de Inteligencia Artificial

- **Herramienta:** Asistente IA (Gemini).
- **Consulta realizada:**
  1. Resolución técnica del error de Maven Fatal error compiling: error: release version 21 not supported y UnsupportedClassVersionError: ... class file version 65.0 ... recognizes class file versions up to 61.0.
  2. Procedimiento para diagnosticar y configurar la variable de entorno JAVA_HOME hacia la instalación de OpenJDK Temurin 21 en Windows PowerShell.
  3. Guía de verificación de endpoints REST y Actuator bajo perfiles default y dev.

     PDF+ 1

- **Verificación realizada:**

  Se validaron las instrucciones en la consola ejecutando los comandos (Get-Command java).Source y .\mvnw.cmd -v, contrastando las respuestas contra la documentación oficial de Spring Boot y el temario del laboratorio. Se comprobó experimentalmente la persistencia de las configuraciones y el correcto despliegue de los puertos 8080 y 8081.

  PDF+ 3

5. Conclusiones Técnicas Registradas

- El mecanismo de perfiles de Spring Boot (application.properties frente a application-dev.properties) permite el aislamiento riguroso de parámetros operacionales (como puertos y mensajes de entorno) sin alterar el código binario compilado.
- Spring Boot Actuator proporciona una observabilidad estandarizada mediante endpoints nativos (/actuator/health y /actuator/info), facilitando la integración con sondas de preparación y vitalidad esenciales para la monitorización de servicios.
- La dependencia Spring Boot DevTools optimiza el flujo de desarrollo al reducir el tiempo de reinicio a través del hilo restartedMain, acelerando la verificación de modificaciones en caliente.
- La correcta configuración de JAVA_HOME y el alineamiento estricto entre el JDK del compilador y el JRE de ejecución evitan inconsistencias de compatibilidad a nivel de bytecode (class file version 65.0).